

FusedAddRmsNorm 与昇腾 NPU：从算子到硬件

目 录

1 引言：为什么要做昇腾算子优化	2
2 FusedAddRmsNorm 算子	2
2.1 RMSNorm 是什么	2
2.2 为什么要融合	2
3 昇腾 910 NPU 与达芬奇架构	3
3.1 达芬奇架构的三种计算单元	3
3.2 AIC / AIV 分离架构	3
4 AIV 内部结构	4
4.1 三条流水线	4
4.2 两种同步机制	4
5 三种算子开发路径	4
5.1 一个完整 Ascend C 算子的组成	5
5.2 多种实现路径的取舍	5
6 本实验的关注点	5
7 本章你将学会	6
8 要点速查	6
9 小结	6

1 引言：为什么要做昇腾算子优化

在前面的实验中，你已经熟悉了 x86-64、ARM 以及 RISC-V 等 CPU 架构，也用过 NVIDIA GPU 做向量化与 kernel 优化。本实验把视野扩展到国产计算生态：你将在昇腾（Ascend）910B4 NPU 上实现并优化一个 fused_add_rmsnorm 算子。算子本身不复杂，但它足以暴露 NPU 编程模型与 GPU 的差异，是一块好的“试金石”。

直觉 不妨把算子想象成一道考题，硬件是考场。同样的题目（残差加法加归一化），在 NVIDIA GPU 考场里你可能驾轻就熟，但换到昇腾 NPU 考场，座次排布、答题工具、交卷规则都变了。本章先带你认考场、读考题，再讲答题套路。

本章我们先认识要优化的算子 FusedAddRmsNorm，再走进昇腾 910 NPU 的达芬奇架构，理解 AIC/AIV 分离设计与 AIV 内部的存储、搬运和计算单元，最后比较三种算子开发路径。

2 FusedAddRmsNorm 算子

2.1 RMSNorm 是什么

RMSNorm（Root Mean Square Normalization，均方根归一化）是一种比 LayerNorm 更轻量的归一化方法。对一个长度为 H 的向量 x ，它先用均方根做缩放，再乘以可学习权重 w ：

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{H} \sum_{i=1}^H x_i^2 + \varepsilon}} \cdot w \quad (2.1)$$

其中 w 是与 x 逐元素相乘的可学习缩放权重， ε 是为数值稳定加入的小常数。与 LayerNorm 相比，RMSNorm 省去了减均值的步骤，计算更轻，在 Transformer 等模型中被广泛用作归一化层。

2.2 为什么要融合

在 Transformer 前向中，RMSNorm 之前往往紧跟一个**残差加法**（Residual Add），把上一路的输入累加到残差流上，这其实就是一个向量加法。如果分开实现，残差结果要先写回全局显存，再由下一个算子读回，多一次 GM 往返。fused_add_rmsnorm 就是把残差加法与 RMSNorm **融合成一个算子**，让中间结果直接留在片上缓冲区，避免多余的显存读写。

融合后的计算为：

$$R = x + \text{residual} \quad (2.2)$$

$$\text{rms} = \sqrt{\frac{1}{H} \sum_{i=1}^H R_i^2 + \varepsilon} \quad (2.3)$$

$$y = \frac{R}{\text{rms}} \cdot w \quad (2.4)$$

算子同时输出两部分： $\text{residual_out} = R$ ，供后续残差流继续使用； $y = \text{RMSNorm}(R)$ ，作为本层的归一化输出。

直觉 不妨把融合想象成做一道“番茄炒蛋”。分开做就是先炒蛋、装盘、再炒番茄、装盘、最后合在一起，要洗两次锅、多两次装盘。融合就是一锅出：蛋炒好不盛出，直接下番茄，中间结果（蛋）留在锅里（片上缓冲），省掉来回搬运。

3 昇腾 910 NPU 与达芬奇架构

3.1 达芬奇架构的三种计算单元

昇腾 NPU 的核心 IP 是**达芬奇架构**（Da Vinci Architecture），它采用 **Cube + Vector + Scalar** 三种计算单元的异构组合：

1. **Cube 单元**：承担矩阵乘法（GEMM）类运算，一个周期可完成一次 $M \times K \times N$ 的矩阵乘（具体维度由代际决定），类似 CPU 上的 AMX/SME 扩展。
2. **Vector 单元**：承担向量与元素级运算（Cast、Add、Mul、Reduce、Sqrt 等），类似 CPU 上的 AVX/SVE 向量化扩展，但宽度较宽（一次处理 256B）。
3. **Scalar 单元**：承担标量运算、地址计算与控制流。

Cube + Vector + Scalar 三个物理单元在一个 **AI Core** 内部并行执行，配合多级片上存储（UB / L1 / L0A / L0B / L0C）完成计算。多个 AI Core 组成一颗 NPU，通过 HBM 全局显存共享数据。

本实验的算子只涉及向量与元素级运算，不涉及矩阵乘法，因此你主要和 AIV（Vector 核）打交道，AIC（Cube 核）相关内容只需了解即可。

3.2 AIC / AIV 分离架构

在 910B 系列上（包括 A2 推理和训练系列），AI Core 采用了 **AIC/AIV 分离** 的设计：

1. **AIC（AI Cube）**：一个 AIC 核内置 Cube 单元、L1、L0A/L0B/L0C 等存储，主要承担矩阵乘类指令。
2. **AIV（AI Vector）**：一个 AIV 核内置 Vector 单元、UB（Unified Buffer）等存储，主要承担向量与元素级指令。
3. AIC 与 AIV 在物理上分离，通过片内总线通信；它们之间以及 AI Core 之间通过全局显存（Global Memory，GM / HBM）协作。

直觉 不妨把 AIC 想成“矩阵乘法专用车间”，AIV 想成“向量运算专用车间”。在分离架构下两个车间物理上分开，各自有独立仓库（AIC 有 L1/L0，AIV 有 UB），要协作就得通过公共仓库（GM）转运。这与 GPU 把 Tensor Core 和 CUDA Core 放在同一 SM 内、共享 Shared Memory 的耦合设计不同。

也存在 AIC/AIV 耦合架构，即可以直接在 AIV 的 UB 与 AIC 的 L1/L0C 之间搬运数据而不必经过 GM，常见于边缘推理系列（如昇腾 310B）。

4 AIV 内部结构

本算子不涉及 Cube 单元，我们重点看 AIV 核内的关键资源（基于 Ascend 910B4 NPU）：

资源	容量/规格	用途
UB（Unified Buffer）	192 KiB / AIV	向量计算的快速访存区, 所有 Vector 指令的操作数必须在 UB 上
Vector 单元	256B (VLEN)	执行 Cast/Add/Mul/Reduce 等向量指令
MTE2 搬运单元	-	GM → UB 的数据搬入
MTE3 搬运单元	-	UB → GM 的数据搬出
Scalar 单元 (S)	-	标量运算, 地址计算, 控制流

直觉 不妨把 AIV 核想成一个小厨房：UB 是操作台（切菜必须在操作台上），Vector 单元是刀（做切、拌、称重），MTE2 是进货口（从仓库往操作台搬料），MTE3 是出货口（从操作台往仓库送成品），Scalar 单元是厨师的脑子（算地址、管流程）。你的任务是让进货、切菜、出货三条线尽量同时跑起来。

4.1 三条流水线

AIV 上有 MTE2（GM → UB）、V（Vector 计算）、MTE3（UB → GM）三条独立的流水线。因为它们物理上独立，如果安排得当，搬运与计算可以**重叠执行**，用一条流水线的时间掩盖另一条流水线的等待，这正是后续优化的核心空间。

4.2 两种同步机制

- 1. **TQue 队列语义自动同步**：当你用 `inQueX.AllocTensor → DataCopy → inQueX.Enqueue` 发射数据搬入 UB 的指令后，再 `inQueX.DeQueue` 取出来给 Vector 用时，`EnQueue/DeQueue` 内部会自动插入 `MTE2 → V` 的同步事件，你不需要手动 `SetFlag/WaitFlag<HardEvent::MTE2_V>`。同理，输出端 `outQueY.Enqueue → outQueY.DeQueue` 也会自动处理 `V → MTE3` 的同步。
- 2. **PipeBarrier 显式屏障**：用于保证某一条流水线内部的同步关系，常用的是 `PIPE_V`。Ascend NPU 不保证 V 流水线内部的 **RAW 依赖**（Read After Write，先写后读）能被自动处理，开发者需要在存在 RAW 依赖的指令之间显式调用 `PipeBarrier<PIPE_V>()`，确保此前发出的所有 V 指令均已执行完成。

Bisheng 编译器提供了 `--cce-auto-sync` 选项，启用后会根据可见的 *LocalTensor* 读写依赖自动插入必要的屏障，减轻编程负担。但当代码涉及指针运算、容器传递或手工地址操作时，编译器的依赖分析可能失效，此时仍需手动插屏障。

5 三种算子开发路径

本实验框架支持 **Ascend C**、**TileLang**、**Triton** 三种算子开发路径。总体而言，Ascend C 更接近昇腾硬件，开发者需要显式处理数据切分、片上存储、数据搬运和计算流水；TileLang 与 Triton 则提供更高层的 DSL 编程模型，由编译器承担更多底层实现工作。

路径	编程模型	对硬件的控制力	与 GPU 经验的相似度
Ascend C	原生 C/C++ 模板 + intrinsic, 显式 tiling/搬运/流水	最强	低, 接近裸金属
TileLang	高层 DSL, tiling/shared memory/ pipeline 作为语言原语	中等	中, 类似 Triton
Triton	block 编程模型, tl.program_id 切分任务	较弱	高, 接近 GPU 经验

直觉 不妨把三种路径想成三种“做菜方式”：Ascend C 是自己种菜、自己切、自己炒，控制力最强但最累；TileLang 是半成品加工，别人切好你炒，省力但口味受限；Triton 是点预制菜包加热，最省事但最难定制。三种方式最终端出的都是一盘菜（跑在 NPU 上的机器码），性能上限由硬件决定，差异只在于“你能在多大程度上接近硬件”。

5.1 一个完整 Ascend C 算子的组成

一个完整的 Ascend C 自定义算子通常由三部分组成：

1. **算子原型（Op 定义）**：声明算子的输入、输出与属性，注册推理 shape 与数据类型的推导函数，以及 tiling 函数。其中 OpDef 类与注册宏等骨架可由 CANN 的 msopgen 工具从算子信息 JSON 自动生成，但 tiling 函数中的切分逻辑仍需开发者手写。
2. **Host 侧 tiling**：在 Host 上根据输入 shape、数据类型和平台信息，计算切分参数，决定每个核处理多少数据、UB 如何分配，并把 tiling 数据传给 kernel。这类似于 CUDA 中设置 grid/block 维度并准备 kernel 参数的 host 侧代码，但额外承担了数据分区与 UB 规划。
3. **Kernel 实现**：在 AI Core 上执行真正的计算，负责 GM $\leftrightarrow$ UB 搬运、UB 内计算、结果写回。这类似于 CUDA 中的 kernel 函数。

5.2 多种实现路径的取舍

你只需要选择一种路径实现即可，**多种实现不会带来进一步加分**。同时，为了让大家接触和了解国产计算生态，课程**鼓励**使用 Ascend C 完成算子。代码框架中只针对 Ascend C 路径提供基线实现，实验文档也以 Ascend C 算子开发为主。但需要注意，使用 Ascend C 路径**不会自动带来额外加分**。

如果你已有 GPU 上的 Triton 经验，Triton-Ascend 路径上手最快；如果你想深入理解昇腾硬件并榨干性能，Ascend C 是更合适的选择。

6 本实验的关注点

本实验的核心目标是在**不改变计算语义的前提下，尽可能提高算子性能**。这意味着：

1. 优化可以涉及 tiling、片上存储布局、数据搬运、计算流水、规约实现等多个层面。
2. 优化的底线是通过正确性检查，任何低精度或近似方案都必须通过精度校验。
3. 优化决策应基于 profiling 证据，而不是凭直觉猜测瓶颈。

后续章节将分别讨论 Ascend C 编程模型、性能分析工具、优化策略与构建运行，逐步展开如何把这个算子跑得更快。

7 本章你将学会

1. 解释 FusedAddRmsNorm 算子的计算语义（残差加法 + RMSNorm），说明融合的价值（减少 GM 往返）。
2. 描述达芬奇架构的 Cube/Vector/Scalar 三种计算单元及其职责。
3. 说明 AIC/AIV 分离架构的特点，对比昇腾 NPU 与 NVIDIA GPU 在设计理念上的异同。
4. 列出 AIV 核内的关键资源（UB、Vector、MTE2、MTE3、Scalar）及其容量与用途。
5. 说明 MTE2/V/MTE3 三条流水线的独立性，以及 TQue 自动同步与 PipeBarrier 显式屏障两种同步机制。
6. 区分 Ascend C、TileLang、Triton 三种开发路径在控制力与上手难度上的差异。

8 要点速查

概念	要点
RMSNorm	$\frac{x}{\sqrt{\frac{1}{H} \sum x_i^2 + \epsilon}} \cdot w$, 省去减均值
FusedAddRmsNorm	残差加法 + RMSNorm 融合, 中间结果 R 留片上, 输出 y 和 <code>residual_out</code>
达芬奇架构	Cube (矩阵乘) + Vector (向量) + Scalar (标量) 异构组合
AIC/AIV 分离	AIC 管 Cube + L1/L0, AIV 管 Vector + UB, 经 GM 协作
AIV 资源	UB 192 KiB, Vector 256B VLEN, MTE2 搬入, MTE3 搬出
三流水线	MTE2 ($\rightarrow$ UB), V (计算), MTE3 ($\rightarrow$ GM), 可重叠
TQue 同步	EnQue/DeQue 自动插入 MTE2 $\rightarrow$ V / V $\rightarrow$ MTE3 同步
PipeBarrier	显式屏障, 处理 V 流水线内部 RAW 依赖, 常用 <code>PIPE_V</code>
三种路径	Ascend C (最强控制), TileLang (DSL), Triton (block 模型)
算子三部分	Op 定义, Host tiling, Device kernel

9 小结

本章从“为什么要做昇腾算子优化”出发，介绍了 FusedAddRmsNorm 算子的计算语义与融合价值，走进达芬奇架构的 Cube/Vector/Scalar 三种计算单元，理解了 AIC/AIV 分离设计与 AIV 核内的 UB、Vector、MTE2/MTE3 等关键资源，以及 TQue 自动同步与 PipeBarrier 显式屏障两种同步机制。最后我们比较了 Ascend C、TileLang、Triton 三种开发路径。后续章节将展开 Ascend C 编程模型的具体细节，带你读懂并改进 baseline 代码。

讲义基于 HPC101 2025 Lab3.5 实验内容编写